

PyMPDATA: an open-source, example-rich, just-in-time compiled implementation of MPDATA finite-difference scheme

Sylwester Arabas

AGH University of Krakow

ESCO 2026 Software Afternoon



- **MPDATA scheme and its implementations**
-

MPDATA key concepts: UPWIND discretisation

- advection equation / scalar conservation law:

$$\partial_t(G\psi) + \nabla \cdot (\mathbf{v}\psi) = GR$$

$\psi(\mathbf{x}, t)$: advected scalar field (advectee),

$\mathbf{v} = \{u, \dots\} = G\dot{\mathbf{x}}$: flow velocity vector field (advectee),

$G(\mathbf{x})$: fluid density, Jacobian of coordinate transformation, or their product

MPDATA key concepts: UPWIND discretisation

- advection equation / scalar conservation law:

$$\partial_t(G\psi) + \nabla \cdot (\mathbf{v}\psi) = GR$$

$\psi(\mathbf{x}, t)$: advected scalar field (advectee),

$\mathbf{v} = \{u, \dots\} = G\dot{\mathbf{x}}$: flow velocity vector field (advecting),

$G(\mathbf{x})$: fluid density, Jacobian of coordinate transformation, or their product

- UPWIND discretisation on a staggered grid (n numbers time steps, i grid steps):

$$\frac{\psi_i^{n+1} - \psi_i^n}{\Delta t} + \frac{\overbrace{f(\psi_i^n, \psi_{i+1}^n, u_{i+1/2}^n)}^{\text{right-hand wall flux}} - \overbrace{f(\psi_{i-1}^n, \psi_i^n, u_{i-1/2}^n)}^{\text{left-hand wall flux}}}{\Delta x} = 0$$

MPDATA key concepts: UPWIND discretisation

- advection equation / scalar conservation law:

$$\partial_t(G\psi) + \nabla \cdot (\mathbf{v}\psi) = GR$$

$\psi(\mathbf{x}, t)$: advected scalar field (advectee),

$\mathbf{v} = \{u, \dots\} = G\dot{\mathbf{x}}$: flow velocity vector field (advector),

$G(\mathbf{x})$: fluid density, Jacobian of coordinate transformation, or their product

- UPWIND discretisation on a staggered grid (n numbers time steps, i grid steps):

$$\frac{\psi_i^{n+1} - \psi_i^n}{\Delta t} + \frac{\overbrace{f(\psi_i^n, \psi_{i+1}^n, u_{i+1/2}^n)}^{\text{right-hand wall flux}} - \overbrace{f(\psi_{i-1}^n, \psi_i^n, u_{i-1/2}^n)}^{\text{left-hand wall flux}}}{\Delta x} = 0$$
$$\partial_t \psi|_i^n + u \partial_x \psi|_i^n = \underbrace{\left(|u| \frac{\Delta x}{2} - u^2 \frac{\Delta t}{2} \right)}_{k - \text{numerical diffusion}} \partial_x^2 \psi|_i^n$$

(e.g., Roberts & Weiss 1966, doi:10.2307/2003507)

MPDATA key concepts: antidiffusive pseudo-velocities

- diffusion can be cast as advection with a pseudo-velocity:

$$\partial_t \psi = k \partial_x^2 \psi + \dots \quad \rightsquigarrow \quad \partial_t \psi = \partial_x \left(\underbrace{k \frac{\partial_x \psi}{\psi}}_{\text{pseudo-velocity}} \psi \right) + \dots$$

(e.g., Lange 1973, doi:10.2172/4308175)

MPDATA key concepts: antidiffusive pseudo-velocities

- diffusion can be cast as advection with a pseudo-velocity:

$$\partial_t \psi = k \partial_x^2 \psi + \dots \quad \rightsquigarrow \quad \partial_t \psi = \partial_x \left(\underbrace{k \frac{\partial_x \psi}{\psi}}_{\text{pseudo-velocity}} \psi \right) + \dots$$

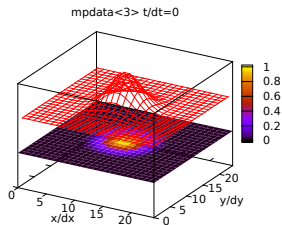
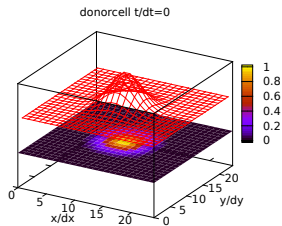
(e.g., Lange 1973, doi:10.2172/4308175)

- “Smolarkiewicz algorithm” (MPDATA): upwind-integrate backwards-in-time, with an anti-diffusive pseudo velocity to reverse the effects of numerical diffusion, iteratively (m numbers iteration)

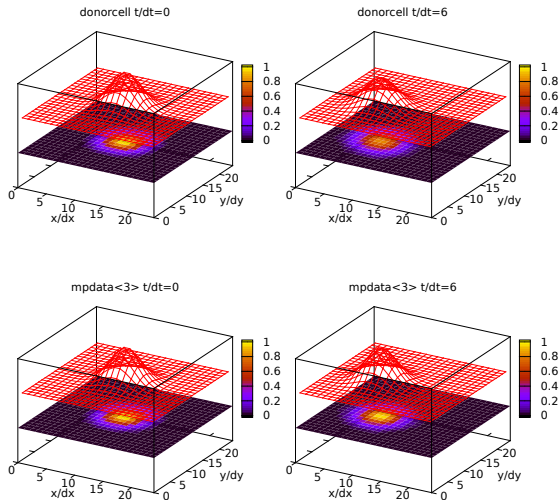
$$C_{i-1/2}^{m+1} = \frac{\Delta t}{\Delta x} k_{i-1/2}^m \left. \frac{\partial_x \psi}{\psi} \right|_{i-1/2}^m \approx \begin{cases} 0 & \text{if } \psi_i^m + \psi_{i-1}^m = 0 \\ \left[|C_{i-1/2}^m| - (C_{i-1/2}^m)^2 \right] \frac{\psi_i^m - \psi_{i-1}^m}{\psi_i^m + \psi_{i-1}^m} & \text{otherwise} \end{cases}$$

(Smolarkiewicz 1983 MWR, 1984 JCP: doi:10.1016/0021-9991(84)90121-9)

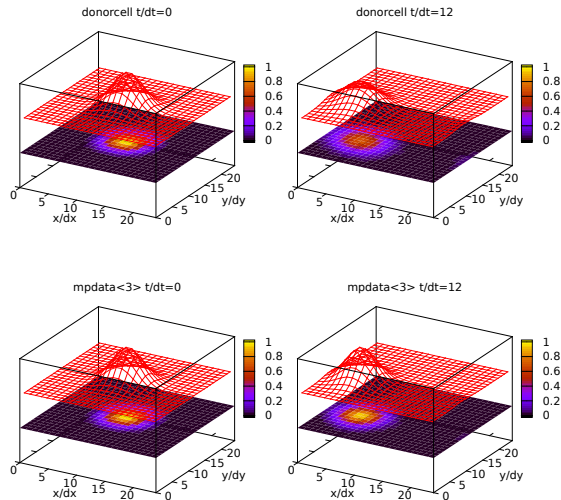
MPDATA: "M" for multi-dimensional (2D is not a sum of two 1D MPDATAs)



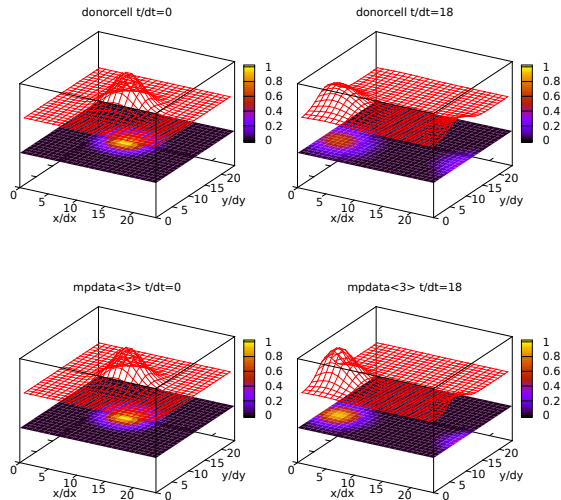
MPDATA: "M" for multi-dimensional (2D is not a sum of two 1D MPDATAs)



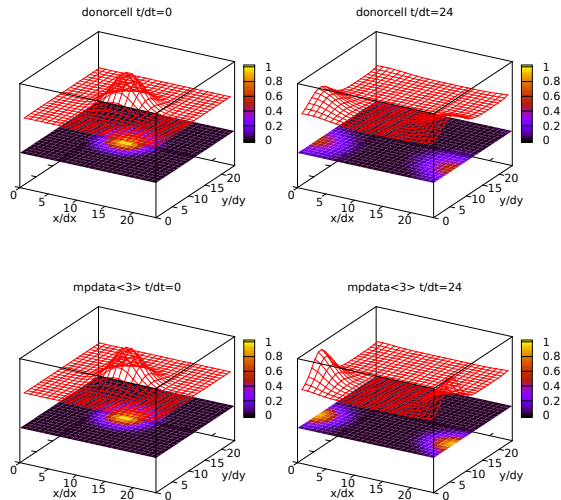
MPDATA: "M" for multi-dimensional (2D is not a sum of two 1D MPDATAs)



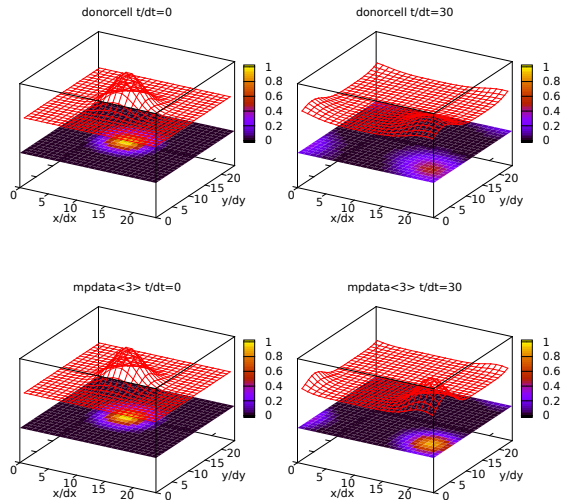
MPDATA: "M" for multi-dimensional (2D is not a sum of two 1D MPDATAs)



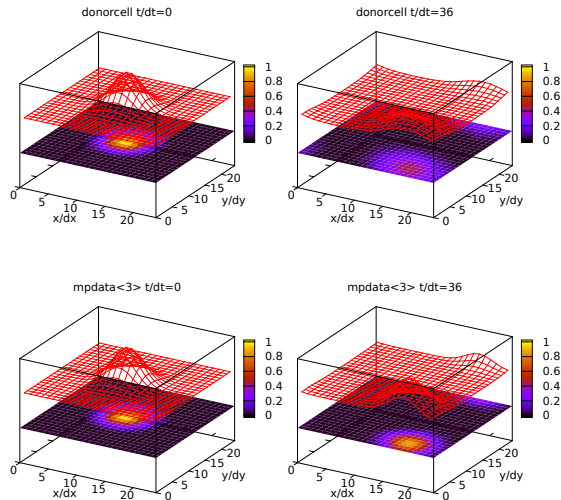
MPDATA: "M" for multi-dimensional (2D is not a sum of two 1D MPDATAs)



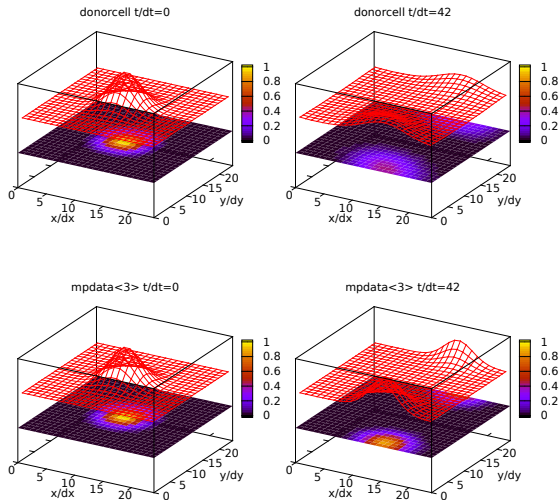
MPDATA: "M" for multi-dimensional (2D is not a sum of two 1D MPDATAs)



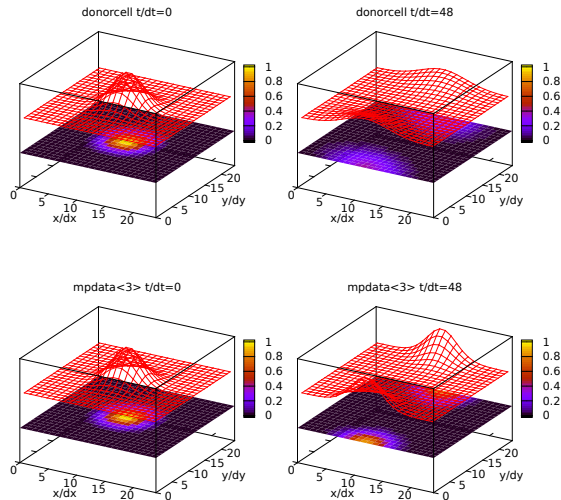
MPDATA: "M" for multi-dimensional (2D is not a sum of two 1D MPDATAs)



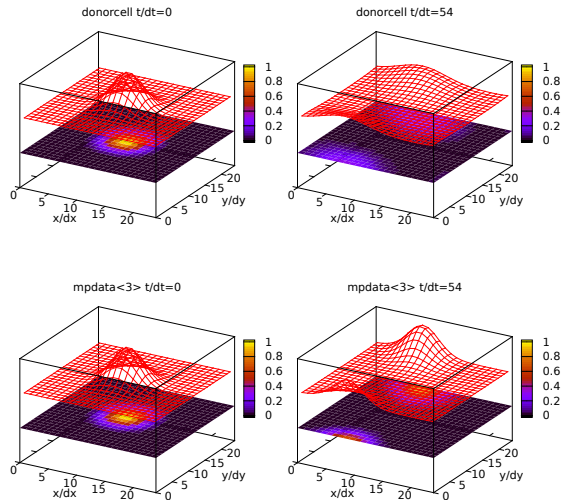
MPDATA: "M" for multi-dimensional (2D is not a sum of two 1D MPDATAs)



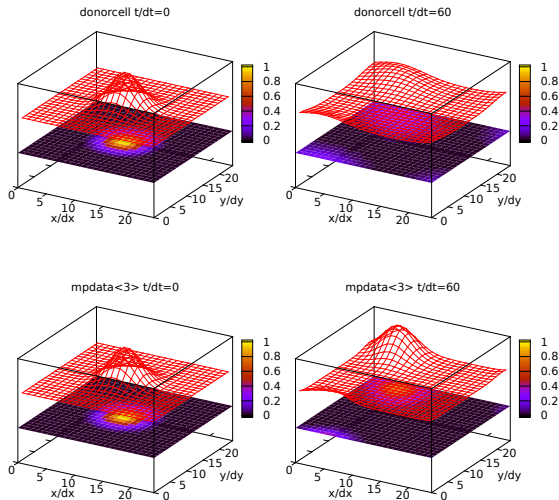
MPDATA: "M" for multi-dimensional (2D is not a sum of two 1D MPDATAs)



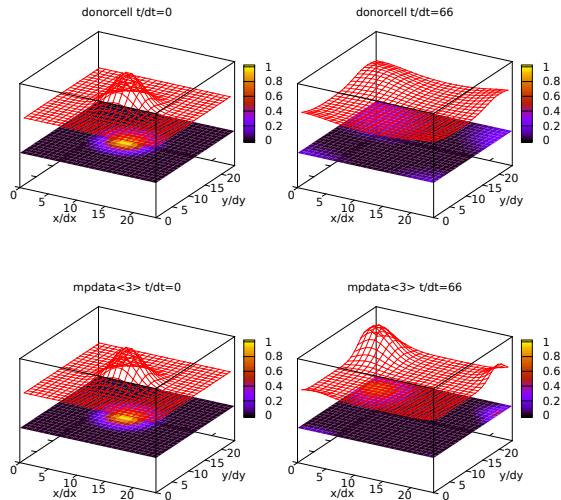
MPDATA: "M" for multi-dimensional (2D is not a sum of two 1D MPDATAs)



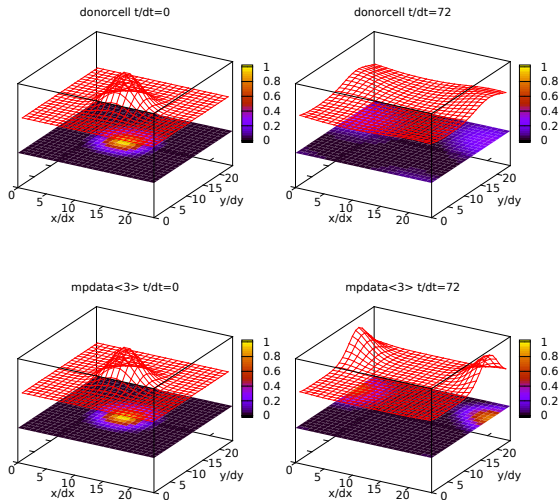
MPDATA: "M" for multi-dimensional (2D is not a sum of two 1D MPDATAs)



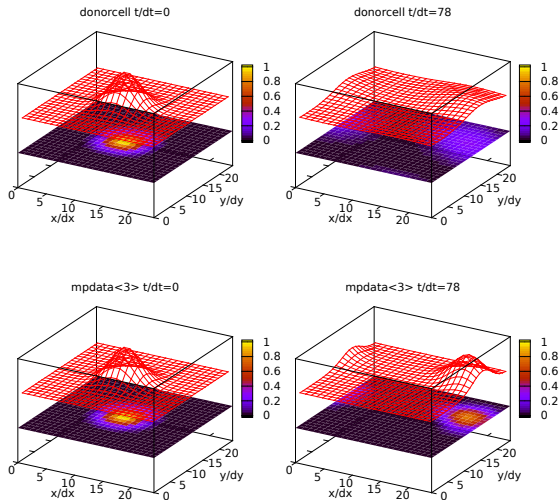
MPDATA: "M" for multi-dimensional (2D is not a sum of two 1D MPDATAs)



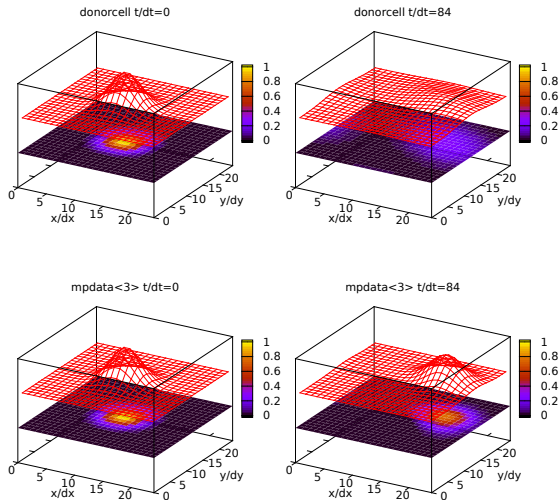
MPDATA: "M" for multi-dimensional (2D is not a sum of two 1D MPDATAs)



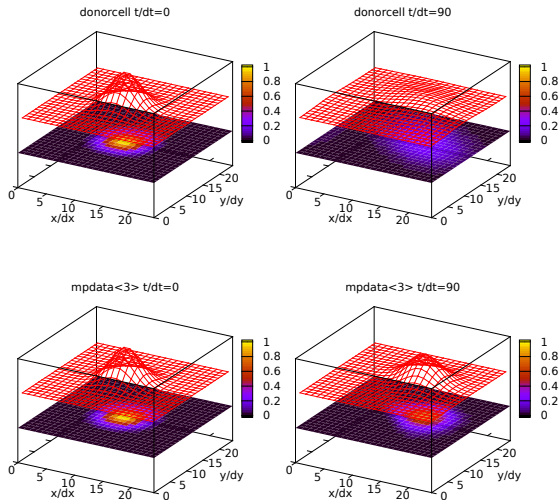
MPDATA: "M" for multi-dimensional (2D is not a sum of two 1D MPDATAs)



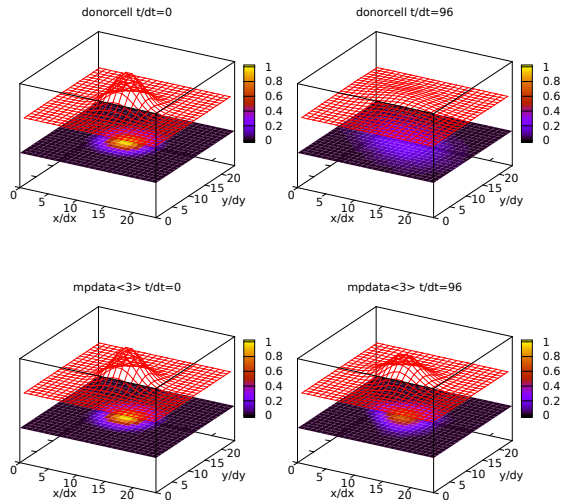
MPDATA: "M" for multi-dimensional (2D is not a sum of two 1D MPDATAs)



MPDATA: "M" for multi-dimensional (2D is not a sum of two 1D MPDATAs)



MPDATA: "M" for multi-dimensional (2D is not a sum of two 1D MPDATAs)



MPDATA variants (structured grid)

- basic (+iterations): Smolarkiewicz 1983

MPDATA variants (structured grid)

- basic (+iterations): Smolarkiewicz 1983
- n-dimensions, diffusion, divergent flow corrections: Smolarkiewicz 1984

MPDATA variants (structured grid)

- basic (+iterations): Smolarkiewicz 1983
- n-dimensions, diffusion, divergent flow corrections: Smolarkiewicz 1984
- coordinate transformation: Smolarkiewicz and Clark 1986, Smolarkiewicz and Margolin 1993

MPDATA variants (structured grid)

- basic (+iterations): Smolarkiewicz 1983
- n-dimensions, diffusion, divergent flow corrections: Smolarkiewicz 1984
- coordinate transformation: Smolarkiewicz and Clark 1986, Smolarkiewicz and Margolin 1993
- resursive anti-diffusive velocities: Beason & Margolin 1988

MPDATA variants (structured grid)

- basic (+iterations): Smolarkiewicz 1983
- n-dimensions, diffusion, divergent flow corrections: Smolarkiewicz 1984
- coordinate transformation: Smolarkiewicz and Clark 1986, Smolarkiewicz and Margolin 1993
- resursive anti-diffusive velocities: Beason & Margolin 1988
- flux-corrected transport: Smolarkiewicz and Grabowski 1990

MPDATA variants (structured grid)

- basic (+iterations): Smolarkiewicz 1983
- n-dimensions, diffusion, divergent flow corrections: Smolarkiewicz 1984
- coordinate transformation: Smolarkiewicz and Clark 1986, Smolarkiewicz and Margolin 1993
- resursive anti-diffusive velocities: Beason & Margolin 1988
- flux-corrected transport: Smolarkiewicz and Grabowski 1990
- third-order terms: Smolarkiewicz and Margolin 1998

MPDATA variants (structured grid)

- basic (+iterations): Smolarkiewicz 1983
- n-dimensions, diffusion, divergent flow corrections: Smolarkiewicz 1984
- coordinate transformation: Smolarkiewicz and Clark 1986, Smolarkiewicz and Margolin 1993
- resursive anti-diffusive velocities: Beason & Margolin 1988
- flux-corrected transport: Smolarkiewicz and Grabowski 1990
- third-order terms: Smolarkiewicz and Margolin 1998
- infinite-gauge variant: Smolarkiewicz 2006

MPDATA variants (structured grid)

- basic (+iterations): Smolarkiewicz 1983
- n-dimensions, diffusion, divergent flow corrections: Smolarkiewicz 1984
- coordinate transformation: Smolarkiewicz and Clark 1986, Smolarkiewicz and Margolin 1993
- resursive anti-diffusive velocities: Beason & Margolin 1988
- flux-corrected transport: Smolarkiewicz and Grabowski 1990
- third-order terms: Smolarkiewicz and Margolin 1998
- infinite-gauge variant: Smolarkiewicz 2006
- fully third-order variant: Waruszewski et al. 2018

MPDATA implementations

known closed-source:

- NCAR EULAG
- COSMO (alternative dynamical core)
- ECMWF IFS-FVM

open-source:

integrated into CFD packages:

MPDATA implementations

known closed-source:

- NCAR EULAG
- COSMO (alternative dynamical core)
- ECMWF IFS-FVM

open-source:

integrated into CFD packages:

AtmosFOAM	C++ & Python/Numba	3D	 /AtmosFOAM	U. Reading
-----------	--------------------	----	--	------------

MPDATA implementations

known closed-source:

- NCAR EULAG
- COSMO (alternative dynamical core)
- ECMWF IFS-FVM

open-source:

integrated into CFD packages:

AtmosFOAM	C++ & Python/Numba	3D	 /AtmosFOAM	U. Reading
ROMS	FORTTRAN	3D	 /myroms	UCLA (?)

MPDATA implementations

known closed-source:

- NCAR EULAG
- COSMO (alternative dynamical core)
- ECMWF IFS-FVM

open-source:

integrated into CFD packages:

AtmosFOAM	C++ & Python/Numba	3D	 /AtmosFOAM	U. Reading
ROMS	FORTTRAN	3D	 /myroms	UCLA (?)
PISM	C++	2D	 /pism	U. Alaska Fairbanks

MPDATA implementations

known closed-source:

- NCAR EULAG
- COSMO (alternative dynamical core)
- ECMWF IFS-FVM

open-source:

integrated into CFD packages:

AtmosFOAM	C++ & Python/Numba	3D	 /AtmosFOAM	U. Reading
ROMS	FORTTRAN	3D	 /myroms	UCLA (?)
PISM	C++	2D	 /pism	U. Alaska Fairbanks
babyEULAG	FORTTRAN	2D	 /igfuw/bE_SDs	NCAR

MPDATA implementations

known closed-source:

- NCAR EULAG
- COSMO (alternative dynamical core)
- ECMWF IFS-FVM

open-source:

integrated into CFD packages:

AtmosFOAM	C++ & Python/Numba	3D	 /AtmosFOAM	U. Reading
ROMS	FORTTRAN	3D	 /myroms	UCLA (?)
PISM	C++	2D	 /pism	U. Alaska Fairbanks
babyEULAG	FORTTRAN	2D	 /igfuw/bE_SDs	NCAR
apc-llc/mpdata	C/CUDA	3D	 /apc-llc	Rus. Acad. Sci.

MPDATA implementations

known closed-source:

- NCAR EULAG
- COSMO (alternative dynamical core)
- ECMWF IFS-FVM

open-source:

integrated into CFD packages:

AtmosFOAM	C++ & Python/Numba	3D	 /AtmosFOAM	U. Reading
ROMS	FORTTRAN	3D	 /myroms	UCLA (?)
PISM	C++	2D	 /pism	U. Alaska Fairbanks
babyEULAG	FORTTRAN	2D	 /igfuw/bE_SDs	NCAR
apc-llc/mpdata	C/CUDA	3D	 /apc-llc	Rus. Acad. Sci.
ICAR	FORTTRAN	3D	 /NCAR/icar	NCAR

MPDATA implementations

known closed-source:

- NCAR EULAG
- COSMO (alternative dynamical core)
- ECMWF IFS-FVM

open-source:

integrated into CFD packages:

AtmosFOAM	C++ & Python/Numba	3D	 /AtmosFOAM	U. Reading
ROMS	FORTTRAN	3D	 /myroms	UCLA (?)
PISM	C++	2D	 /pism	U. Alaska Fairbanks
babyEULAG	FORTTRAN	2D	 /igfuw/bE_SDs	NCAR
apc-llc/mpdata	C/CUDA	3D	 /apc-llc	Rus. Acad. Sci.
ICAR	FORTTRAN	3D	 /NCAR/icar	NCAR
sbPOM	FORTTRAN	2D	 /RinceWND/extPOM	Stony Brook

MPDATA implementations

known closed-source:

- NCAR EULAG
- COSMO (alternative dynamical core)
- ECMWF IFS-FVM

open-source:

integrated into CFD packages:

AtmosFOAM	C++ & Python/Numba	3D	 /AtmosFOAM	U. Reading
ROMS	FORTTRAN	3D	 /myroms	UCLA (?)
PISM	C++	2D	 /pism	U. Alaska Fairbanks
babyEULAG	FORTTRAN	2D	 /igfuw/bE_SDs	NCAR
apc-llc/mpdata	C/CUDA	3D	 /apc-llc	Rus. Acad. Sci.
ICAR	FORTTRAN	3D	 /NCAR/icar	NCAR
sbPOM	FORTTRAN	2D	 /RinceWND/extPOM	Stony Brook

reusable:

libmpdata++	C++/Blitz++	1,2,3D	 /igfuw	UWarsaw
-------------	-------------	--------	--	---------

MPDATA implementations

known closed-source:

- NCAR EULAG
- COSMO (alternative dynamical core)
- ECMWF IFS-FVM

open-source:

integrated into CFD packages:

AtmosFOAM	C++ & Python/Numba	3D	 /AtmosFOAM	U. Reading
ROMS	FORTTRAN	3D	 /myroms	UCLA (?)
PISM	C++	2D	 /pism	U. Alaska Fairbanks
babyEULAG	FORTTRAN	2D	 /igfuw/bE_SDs	NCAR
apc-llc/mpdata	C/CUDA	3D	 /apc-llc	Rus. Acad. Sci.
ICAR	FORTTRAN	3D	 /NCAR/icar	NCAR
sbPOM	FORTTRAN	2D	 /RinceWND/extPOM	Stony Brook

reusable:

libmpdata++	C++/Blitz++	1,2,3D	 /igfuw	UWarsaw
PyMPDATA	Python/Numba	1,2,3D	 /open-atmos	UJ/AGH.edu.pl

- **PyMPDATA: pure-Python
just-in-time compiled MPDATA**
-

PyMPDATA: design goals, tech stack, features

- Numba JIT $\rightsquigarrow$ pure-Python code with compiled-language performance (plus OpenMP-like multi-threading, but no profiling tools)



PyMPDATA: 100% Python codebase

```
@numba.njit(**options.jit_flags)
def a_term(psi):
    """eq. 13 in [Smolarkiewicz 1984](https://doi.org/10.1016/0021-9991(84)90121-9);
    eq. 17a in [Smolarkiewicz & Margolin 1998](https://doi.org/10.1006/jcph.1998.5901)"""
    result = ats(*psi, 1) - ats(*psi, 0)
    if infinite_gauge:
        return result / 2
    return result / (ats(*psi, 1) + ats(*psi, 0) + epsilon)

@numba.njit(**options.jit_flags)
def b_term(psi):
    """eq. 13 in [Smolarkiewicz 1984](https://doi.org/10.1016/0021-9991(84)90121-9);
    eq. 17b in [Smolarkiewicz & Margolin 1998](https://doi.org/10.1006/jcph.1998.5901)"""
    result = ats(*psi, 1, 1) + ats(*psi, 0, 1) - ats(*psi, 1, -1) - ats(*psi, 0, -1)
    if infinite_gauge:
        return result / 4

    return result / (
        ats(*psi, 1, 1)
        + ats(*psi, 0, 1)
        + ats(*psi, 1, -1)
        + ats(*psi, 0, -1)
        + epsilon
    )
```

PyMPDATA: design goals, tech stack, features

- Numba JIT $\rightsquigarrow$ pure-Python code with compiled-language performance (plus OpenMP-like multi-threading, but no profiling tools)
- runs on Linux, macOS & Windows (no compilation, just "pip install")



PyMPDATA

PyMPDATA: design goals, tech stack, features



PyMPDATA

- Numba JIT $\rightsquigarrow$ pure-Python code with compiled-language performance (plus OpenMP-like multi-threading, but no profiling tools)
- runs on Linux, macOS & Windows (no compilation, just "pip install")
- > 90% unit-test coverage (codecov) and growing...

PyMPDATA: design goals, tech stack, features



PyMPDATA

- Numba JIT $\rightsquigarrow$ pure-Python code with compiled-language performance (plus OpenMP-like multi-threading, but no profiling tools)
- runs on Linux, macOS & Windows (no compilation, just "pip install")
- > 90% unit-test coverage (codecov) and growing...
- API: 5 classes & single "advance" method

PyMPDATA: design goals, tech stack, features



PyMPDATA

- Numba JIT $\rightsquigarrow$ pure-Python code with compiled-language performance (plus OpenMP-like multi-threading, but no profiling tools)
- runs on Linux, macOS & Windows (no compilation, just "pip install")
- > 90% unit-test coverage (codecov) and growing...
- API: 5 classes & single "advance" method
- array-traversal abstractions avoiding explicit fill-halo or barrier calls

PyMPDATA: design goals, tech stack, features



- Numba JIT $\rightsquigarrow$ pure-Python code with compiled-language performance (plus OpenMP-like multi-threading, but no profiling tools)
- runs on Linux, macOS & Windows (no compilation, just "pip install")
- > 90% unit-test coverage (codecov) and growing...
- API: 5 classes & single "advance" method
- array-traversal abstractions avoiding explicit fill-halo or barrier calls
- docs (and CI) covers usage from Python, Julia, Matlab and Rust

PyMPDATA: design goals, tech stack, features



- Numba JIT $\rightsquigarrow$ pure-Python code with compiled-language performance (plus OpenMP-like multi-threading, but no profiling tools)
- runs on Linux, macOS & Windows (no compilation, just "pip install")
- > 90% unit-test coverage (codecov) and growing...
- API: 5 classes & single "advance" method
- array-traversal abstractions avoiding explicit fill-halo or barrier calls
- docs (and CI) covers usage from Python, Julia, Matlab and Rust
- suite of 20+ Jupyter notebook examples maintained with the project all with badges enabling **single-click execution on Colab**

PyMPDATA: design goals, tech stack, features



- Numba JIT $\rightsquigarrow$ pure-Python code with compiled-language performance (plus OpenMP-like multi-threading, but no profiling tools)
- runs on Linux, macOS & Windows (no compilation, just "pip install")
- > 90% unit-test coverage (codecov) and growing...
- API: 5 classes & single "advance" method
- array-traversal abstractions avoiding explicit fill-halo or barrier calls
- docs (and CI) covers usage from Python, Julia, Matlab and Rust
- suite of 20+ Jupyter notebook examples maintained with the project all with badges enabling **single-click execution on Colab**
- examples in 1D, 2D & 3D: advection-diffusion, bin cloud μ -physics, spherical coordinates, shallow-water, Black-Scholes, Burgers, Boussinesq

- **PyMPDATA "examples" and documentation**
-



Documentation

What is PyMPDATA?

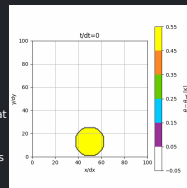
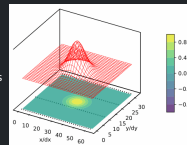
PyMPDATA is a **Numba-accelerated** **multi-threaded** Pythonic implementation of the **MPDATA algorithm** of Smolarkiewicz et al. used in geophysical fluid dynamics and beyond for **numerically solving generalised convection-diffusion PDEs**. PyMPDATA supports integration in 1D, 2D and 3D structured meshes with optional coordinate transformations. The first animation shown depicts a "hello-world" 2D advection-only simulation with dotted lines indicating **domain decomposition** across three threads. The second animation depicts an MPDATA solution to coupled mass and momentum conservation equations for a buoyancy-driven flow in Boussinesq approximation (see Jaruga et al. 2015 example).

A separate project called **PyMPDATA-MPI** depicts how **numba-mpi** can be used to enable **distributed memory parallelism** in PyMPDATA.

What is the difference between PyMPDATA and PyMPDATA-examples?

PyMPDATA is a Python package that provides the MPDATA algorithm implementation. It is a library that can be used in your own projects.

PyMPDATA-examples is a Python package that provides examples of how to use PyMPDATA. It includes common Python modules used in PyMPDATA smoke tests and in example Jupyter notebooks (but the package wheels do not include the notebooks, only .py files imported from the notebooks and PyMPDATA tests).



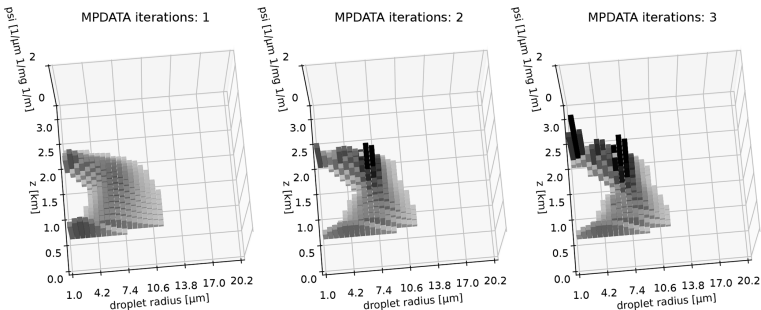
MPDATA for condensational growth in bin μ -physics (Olesik et al. 2022)

Geosci. Model Dev., 15, 3879–3899, 2022
<https://doi.org/10.5194/gmd-15-3879-2022>



On numerical broadening of particle-size spectra: a condensational growth study using PyMPDATA 1.0

Michael A. Olesik¹, Jakub Banaśkiewicz², Piotr Bartman², Manuel Baumgartner^{3,4}, Simon Unterstrasser⁵, and Sylwester Arabas^{6,2}



- spectro-spatial advection (single-column model)
- spectral broadening vs. MPDATA options

MPDATA for Asian option pricing using 2D PDE (Magnuszewski et al. 2025)

arXiv > q-fin > arXiv:2505.24435

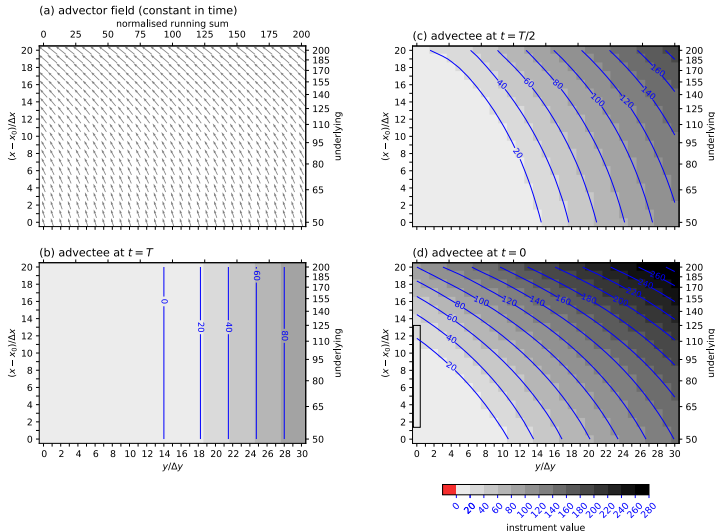
Quantitative Finance > Computational Finance

[Submitted on 30 May 2025]

Path-dependent option pricing with two-dimensional PDE using MPDATA

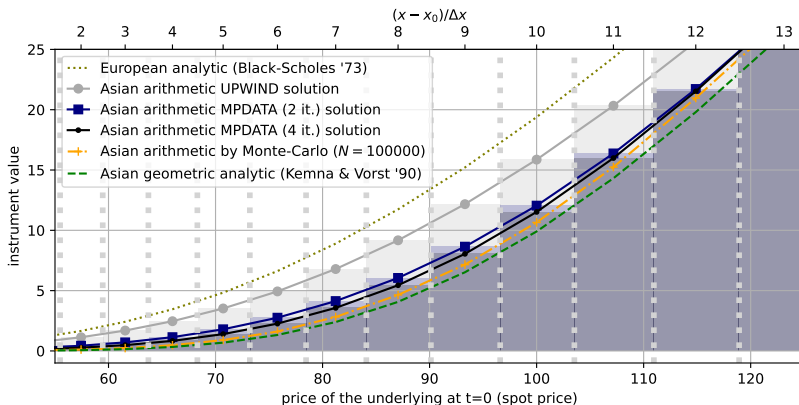
Paweł Magnuszewski, Sylwester Arabas

In this paper, we discuss a simple yet robust PDE method for evaluating path-dependent Asian-style options using the non-oscillatory forward-in-time second-order MPDATA finite-difference scheme. The valuation methodology involves casting the Black-Merton-Scholes equation as a transport problem by first transforming it into a homogeneous advection-diffusion PDE via variable substitution, and then expressing the diffusion term as an advective flux using the pseudo-velocity technique.



MPDATA for Asian option pricing using 2D PDE (Magnuszewski et al. 2025)

$$\frac{\partial f}{\partial t} + rS \frac{\partial f}{\partial S} + \frac{\sigma^2}{2} S^2 \frac{\partial^2 f}{\partial S^2} + v \frac{\partial f}{\partial A} - rf = 0$$



- **demo time!**

(note: wall time incl. JIT compilation)

- **PyMPDATA performance vs. C++**
-



DOI: [10.21105/joss.03896](https://doi.org/10.21105/joss.03896)

Software

- [Review](#) ↗
- [Repository](#) ↗
- [Archive](#) ↗

Editor: [Arfon Smith](#) ↗

Reviewers:

- [@Chil](#)
- [@wdeconinck](#)

Submitted: 25 October 2021

Published: 05 September 2022

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

PyMPDATA v1: Numba-accelerated implementation of MPDATA with examples in Python, Julia and Matlab

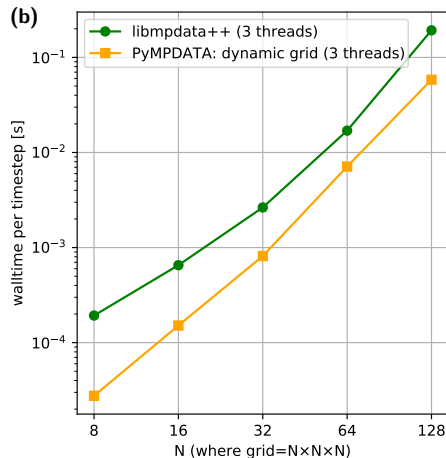
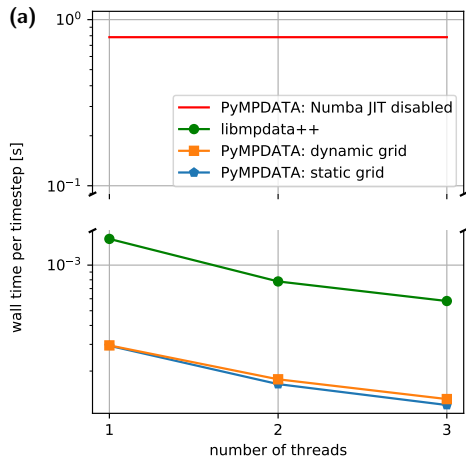
Piotr Bartman¹, Jakub Banaśkiewicz¹, Szymon Drenda¹, Maciej Manna¹, Michael A. Olesik¹, Paweł Rozwoda¹, Michał Sadowski¹, and Sylwester Arabas^{1,2}

¹ Jagiellonian University, Kraków, Poland ² University of Illinois at Urbana-Champaign, IL, USA

Statement of need

Convection-diffusion problems arise across a wide range of pure and applied research, in particular in geosciences, aerospace engineering, and financial modelling (for an overview of applications, see, e.g., section 1.1 in Morton (1996)). One of the key challenges in numerical solutions of problems involving advective transport is sign preservation of the advected field (for an overview of this and other aspects of numerical solutions to advection problems, see, e.g., Røed (2019)). The Multidimensional Positive Definite Advection Transport Algorithm (MPDATA) is a robust, explicit-in-time, and sign-preserving solver introduced in Smolarkiewicz (1983) and Smolarkiewicz (1984). MPDATA has been subsequently developed into a family of numerical schemes with numerous variants and solution procedures addressing a diverse set of problems in geophysical fluid dynamics and beyond. For reviews of MPDATA applications and variants, see, e.g., Smolarkiewicz & Margolin (1998) and Smolarkiewicz (2006).

Numba JIT & multi-threading: PyMPDATA vs. libmpdata++ performance



Bartman et al. 2022 (JOSS, doi:10.21105/joss.03896)

- **MPI, HPC & distributed-memory parallelism**
-



Original software publication

Numba-MPI v1.0: Enabling MPI communication within Numba/LLVM JIT-compiled Python code

Kacper Derlatka ^{a 1}, Maciej Manna ^{a 2}, Oleksii Bulenok ^{a 3}, David Zwicker ^b, Sylwester Arabas ^c  

^a Faculty of Mathematics and Computer Science, Jagiellonian University in Kraków, Poland

^b Max Planck Institute for Dynamics and Self-Organization, Göttingen, Germany

^c Faculty of Physics and Applied Computer Science, AGH University of Krakow, Poland

<https://doi.org/10.1016/j.softx.2024.101897> 

Under a Creative Commons [license](#) 

 Open access

Abstract

The numba-mpi package offers access to the Message Passing Interface (MPI) routines from Python code that uses the Numba just-in-time (JIT) compiler. As a result, high-performance and multi-threaded Python code may utilize MPI communication facilities without leaving the JIT-compiled code blocks, which is not possible with the mpi4py package, a higher-level Python interface to MPI. For debugging or code-coverage analysis purposes, numba-mpi retains full functionality of the code even if the JIT compilation is disabled.



[Help](#) [Docs](#) [Sponsors](#) [Log in](#) [Register](#)

numba-mpi 1.3.1

`pip install numba-mpi` 

 [Latest release](#)

Released: May 27, 2026

Numba @jittable MPI wrappers tested on Linux, macOS and Windows

Navigation

- [Project description](#)
- [Release history](#)
- [Download files](#)

Project description



Numba-MPI

Python

LLVM

Numba

Linux

macOS

Windows

tests+pypi

failing

Maintained?

yes

License

GPL v3

pypi package

1.3.1

Anaconda.org

1.3.1

AUR package

DOI

10.5281/zenodo.20410878

Overview

Numba-MPI provides Python wrappers to the C MPI API callable from within [Numba JIT-compiled code](#) (@jit mode). For an outline of the project, rationale, architecture, and features, refer to: [Numba-MPI paper in SoftwareX \(open access\)](#) (please cite if Numba-MPI is used in your research).

Support is provided for a subset of MPI routines covering: `size/rank`, `send/recv`, `sendrecv`, `allreduce`, `reduce`, `bcast`, `scatter/gather` & `allgather`, `barrier`, `wtime` and basic asynchronous communication with `isend/irecv` (only for contiguous arrays); for request handling including `wait/waitall/waitany` and `test/testall/testany`.

Verified details

 These details have been [verified by PyPI](#)

Project links

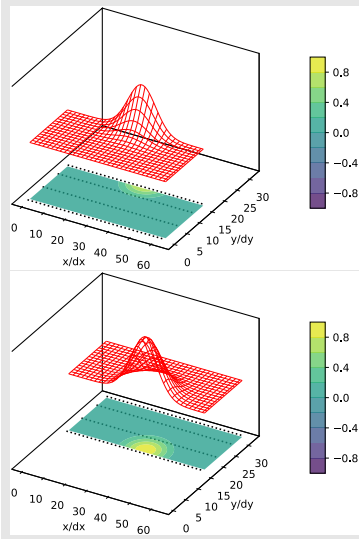
- [Documentation](#)
- [Source](#)
- [Tracker](#)

GitHub Statistics

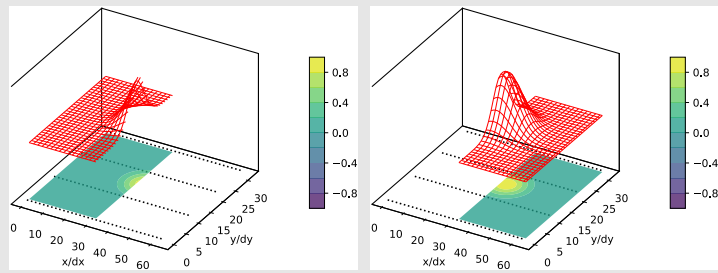
- [Repository](#)
- [Stars: 47](#)

PyMPDATA-MPI: customisable hybrid threading + MPI parallelisation

threading dim = MPI dim



threading dimension $\neq$ MPI dimension



Derlatka et al. 2024 (SoftwareX, doi:10.1016/j.softx.2024.101897)

MPDATA Wikipedia article: contributions welcome!

<https://en.wikipedia.org/wiki/Draft:MPDATA>

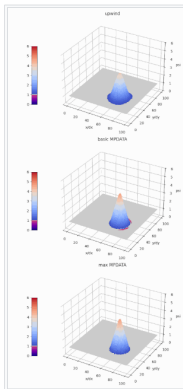
In numerical analysis of partial differential equations, **MPDATA** is a family of *iterative finite-difference/finite-volume* methods for numerical integration of *hyperbolic differential equations* modelling *conservation laws* of the form:

$$\partial_t (G\psi) + \nabla \cdot (\mathbf{v}\psi) = GR, \quad (1)$$

where $\psi(\mathbf{x}, t)$ is an *advected scalar field* (or *advectee*); $\mathbf{v} = G\mathbf{x}$ is the *flow velocity vector field* (or *advector*), $G(\mathbf{x})$ may either play the role of the *fluid density*, the *Jacobian* of *coordinate transformation* from *Cartesian* to *curvilinear* framework $\mathbf{x}$, or their product; $R(\mathbf{x})$ combines all source terms^[1].

MPDATA stands for Multidimensional Positive Definite Advection Transport Algorithm. The algorithm was formulated by Piotr K. Smolarkiewicz^{[2][3]} (and is also referred to as Smolarkiewicz's method^[4]) at the *US National Center for Atmospheric Research - NCAR* (at the time, Smolarkiewicz was a fellow of the NCAR Advanced Study Program and a recent graduate from *Krzysztof Haman's* group at the *University of Warsaw*; the seminal 1983 and 1984 MPDATA publications mention both institutions).

The crux of the method lies in iterative application of the *upwind scheme*. The first iteration employs the advective velocity $\mathbf{v}$, while each subsequent iteration employs a so-called **antidiffusive velocity** which corrects solution from prior iteration reducing the *numerical diffusion*. The antidiffusive velocities can be derived through modified equation analysis^[5] of the *upwind scheme* and feature cross-dimensional dependencies (i.e., applying MPDATA in multiple dimensions is not equivalent to application of one-dimensional MPDATA in all dimensions), the scheme is thus **not dimensionally split**, hence "M" in the algorithm name. Since each iteration of the scheme constitutes a *forward-in-time* upwind pass, the scheme inherits characteristics of the upwind scheme: *CFL stability criterion*, conservativeness, *embarrassingly parallel* domain decomposition,



Comparison of UPWIND (top panel), basic MPDATA (middle panel) and 3-iteration non-oscillatory MPDATA (bottom panel) applied to transport of a cone-shaped signal in a two-dimensional stationary rotational flow ("Molenkamp test")

pypi.org/p/PyMPDATA

pypi.org/p/Numba-MPI

Thanks to all (>20 students) code contributors!

(GitHub stars warmly welcome)

Thank you for your attention!

sylwester.arabas@agh.edu.pl